Reasoning and Computation

Kevin Sullivan

August 25, 2025

Contents

Preface

Acknowledgements

This course was developed with support provided in part by a research grant from the National Science Foundation, #1909414, SHF: Small: Explicating and Exploiting the Physical Semantics of Code.

Disclaimers

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

The views expressed in this article are those of the author(s) and do not necessarily reflect the views, policies, or positions of the University of Virginia.

This work expresses cetain technical juddgments by the author based on observation and experience but not always on outcomes of scientific testing. No IRBs have been needed or sought. No student or other human subjects data is reported here or has been reported outside of official reporting channels.

A Couple of Pillars

Computation and reasoning are two great intertwined pillars of computer science. The definition of a procedure intended to return the sum of any two given non-negative whole numbersinvolves computational thinking, around both data and procedure definitions.

On the other hand, if we want to remove any doubt that addition as defined by our code has some a critical propery of addition from elementary arithmetic, then we must engage in reasoning. First, we must express the property of arithmetic of interest in a mathematical logic. Second, we must express the proposition that the computation defined by our code has that same property. Let's use a concrete example.

One property of addition in elementary arithmetic is that the sum of any number, x, with, respectively, two unequal numbers, y and z, will *always* produce unequal results. We first make the assertion precise by expressing it in a formal logic. In *predicate logic* we could express the claims as $\forall (x\ y\ z : Nat),\ y \neq z \Rightarrow x + y \neq x + z$: for any three natural numbers (Nat), call them x, y, and z, if y is not equal to z then x + y is not equal to x + z*.

Consequently we have languages for expressing *conputations*, namely *programming* languages, and languages for expressing propositions about worlds or domains of interest, namely *logics*, paired with formal languages for expressing rigorous arguments for either the *truth* of such propositions in a specific domain or world, and their *validity*, which is to say their truth in all possible worlds within the given domain of discourse.

For decades, computer science educators have excelled at teaching *computational* thinking and the use of programming languages to express computations. By contrast, we have done much less well teaching reasoning and the formal languages and thought processes needed to reason formally in practice.

The use of programming langauges is familiar territory even to the earliest computer science students. The very first course in computer science is invariably a course in programming, and implicitly in programming languages. Programming then remains one of the primary areas of emphasis throughout the entire undergraduate curriculum. Students of CS thus today generally graduate with high proficiency in computational thinking and in the use of programming languages that support it.

Recognizing the essential foundational character or reasoning, the *second* course in the undergraduate CS curriculum is typically called something like CS2: Discrete Mathematics and Theory (DMT1). It is in this course that students gain their first, and sadly usually their last, exposure to formal reasoning and languages and systems that support it. Such courses are generally paper-and-pencil affairs covering propositional logic, first-order predicate logic and set theory, and induction (but usually only on natural numbers).

The problem with the ubiquity of such courses are many. First, CS students tend ot find these courses boring, abstract, disconnected from their intrinsic motations to learn about computing, irrelevant to practice, and deeply forgettable. Anecdotally, most students entering graduate programs in CS appear remember almost nothing from their early DMT courses, and few have ever had to use reasoning langauges and methods after DMT1.

On the demand side, on the other hand, we are now seeing rapidly growing needs for engineers who do actually understand formal reasoning and languages. On the other hand, the supply of such talent is miniscule, due in large part to the failure to train our students in such knowledge and skills. Moreover, this explosion in demand for reasoning skills is happening at the same time we're seeing a significant drop-off in demand for ”mere” computational thinking and programming.

The conclusions of the author include the following: (1) Our field has failed to train generations of graduating computer scientists in the thought processes and the formal languages needed to be productive with *reasoning* in theory or industrial practice. (2) The standard DMT1 course is, for far too many students, not a productive or memorable experience, as evinced by the exceptionally poor state of knowledge of most incoming graduate students in computer science (in the author's experience). (3) It is time to replace the standard DMT1 course with something entirely new, different, and far better. (4) It is time to think about re-balancing the entire undergraduate curriculum toward greater emphasis on mathematical abstractions and formal reasoning.

The course presented here is thus offered as a model for an entirely new approach. At the highest level, it teaches all of the core material in any DMT1 course but with all of the context formalized in the reasoning language, Lean 4. In languages like this, supported by wonderful tooling, reasoning is linked to comptuation by the amazing unification known as the the *Curry-Howard Correspondence* (CHC). The CHC holds that formalized deductive reasoning of certains kinds (natural deduction, which is perhaps the core concept in any DMT course) is a form of computing, but not only with the usual data and function types but with now axioms, propositions, and proofs as first-class citizens.

Lean 4 is so beautifully expressive of such a broad range of mathematical concepts that a significant community of mathematicians have organized around it to drive the development of formalized versions of mathematics across a very broad range of fields. Meanwhile, CS students remain stuck learning a logic (first-order predicate logic and set theory) that is *not* suitable as a foundation for formalized or automated abstract mathemtics. This course, on the other hand, adopts, *type theory*, here as implemented by Lean 4, as a far better choice, even for early CS students.

Some Problems

That we've arrived at a point where reasoning technology is advancing at extraordinary speed but where are students are by and large entirely unprepared to understand or use it. Of course, for many decades, the demand for programming was voracious, and at the same time cost and difficulty of reasoning were prohibitively high. But now the tables are turned. Generative and related AI promise to reduce demand for programming code while the needs of industry and national security are driving significant increases in demand for formal reasoning.

This course aims to help address the resulting shortfall in talent by radically replacing the traditional undergraduate DMT1 course with a new one, covering essentiall the same basic content, but now using the wildly successful reasoning and computation language and toolset of Lean 4. The course is scoped for a full undergraduate semester or as the first half of an introductory graduate course in formal languages and reasoning. Big changes in in circumstances make now a great time to consider such a transition in CS pedagogy. They include the following:

- Rapidly increasing industrial demand for formal, machine-supported and machine-checked reasoning about critical properties software-intensive systems that undergird our society
- The emergence of type-theory-based formalisms with exceptional expressiveness and broad applications that have attracted large communities of researchers in mathematics, which gtends to validate the proposition that there's something new and remarkable in them
- The development of superb tooling for using reasoning languages effectively in practice
- The profound intertwining of computation and reasoning afforded by such langauges
- The real possibility that mere routine programming will increasingly be done by ”AIs”

New Approach

The idea is to simultaneously gain a deeper understanding of reasoning while also seeing it as a form computation, albeit now over reasoning rather than computational terms. For example, we begin with propositional logic---syntax, semantics, validity, soundness, etc.--through its standard deep embedding into (the logic of) Lean 4. A demonstrated strength of Lean 4 is in its enabling communities to express rich theories in the clear, abstract, generalized terms of the particular domain itself, across a wide range of domains in graduate- and beyond-level mathematics.

The entire course is set up this way. Predicate logic is presented through its standard shallow embedding in Lean 4. First-order logic is described as a special case. Set theory is built directly on predicate logic. Etc.

This course can then express generalized mathematical concepts, such as the property of a relation of being reflexive (or whatever). In a first-order course, you can formally express what it means for a particular binary relation, r, to be ”reflexive.” That's ok. But one really hopes the student will acquire is an understanding of the property itself: the property of any binary relation, r, on any set, s, of being reflexive. This generalized concept can then be *applied* to any particular relation to speak of its being reflexive, or not. The first-order theory of the traditional DMT1 course isn't expressive enough to represent generalized properties of higher-order things, such as relations, functions, or types. Lean is not much harder to learn and is really a better language for expressing and working with core DMT1 concepts. It's really better to learn from the outset to be able to say things like this: reflexivity is a property of *any* binary relation on *any* set of objects of *any* type. None of those *anys* can be said in first-order logic as one cannot quantify over relations, sets, or types.

It's not just a nicety, either, to have *reflexive* as a predicate on any binary relation on any set of terms of any type. It means that this predicate can be applied to any particular relation so as to produce the proposition that it is reflexive. The application of predicates to particulars is ubiquitous in formal reasoning.

Another principle is that all of the main concepts taught in the traditional course must be taught in the new course: propositional logic, predicate logic, sets, induction. This course covers the same topics but in different ways.

But it's not only topic coverage. Notations matter. Embeddings of mathematical concepts in Lean often come with the standard notations of the field as a paper-and-pencil affair. Differences in surface syntax in having to read and write in set theory as embedded in Lean and as learned using paper and pencil are minor, while the gain in capabilities at one's fingertips is substantial. They include automated syntax and proof checking, among other things. Having a superb supportive community of experts is a tremendous human asset.

Design Constraints

This course was developed under a few key constraints:

- Continue to focus on the basic content of the traditional course: logic, sets, induction
- Avoid assiduously overwhelming early students with the complexity of modern proof assistants
- Formalize every concept in the uniform logic of the proof assistant using conventional notations
- Ensure that first-order theory is a special case of the more expressive theory of the course
- Provide students with a deeply computational perspective, from great tooling to Curry-Howard

This Solution

The solution, now tested in practice (but not scientifically evaluated yet), has a few key elements:

- Make standard embeddings propositional and predicate logic in Lean a path to Lean 4 itself
- Begin with a deep embedding of propositional logic syntax, semantics, and validity in Lean 4
- Thoroughly cover all of the axioms for reasoning in predicate logic as its embedded in Lean 4
- Build all of the material on set theory (sets, relations, properties) on top of this logic
- Present induction first as a way to build functions and only later as a way to build proofs
- Minimize covereage of complex or inessential Lean; e.g., tactics are omitted until the end

An Example

Here are two simple examples of what students might see in this course.

The first illustrates how students would write propositional logic expressions.

- def andAssociative := ((P ? Q) ? R) ? (P ? (Q ? R))
- def equivalence := (P ? Q) ? ((P ? Q) ? (Q ? P))

This one, second, specifies the generalized property of a relation of being well founded.

- def isWellFounded {α β : Type} : Rel α α □ Prop := fun r => □ (s : Set α), s ≠ □ □ □ m, (m □ s □ ¬□ n □ s, r n m)

By the end of the course students should be able to read and explain what this definition means, and *apply* it to particulars in the process of making richer claims about them. The undergraduate course does emphasize ongoing practice in the skills of translating between formal and *natural* natural language.

Status

The status of this online book is *drafted work in progress*. The material through the second chapter on induction reflects the content, edited, from my Fall 2024 undergraduate class offering. It's also the first part of an introductory graduate *software logic* course that I'm teaching this Spring of 2025. There are a few concept/chapter placeholders at the end for material not previously taught at all, including formalized reasoning about induction through well founded relations. (Students have learned and demonstrated the abilty to reason formally about properties of relations, though, and should be able to handle proofs involving this property without much more difficulty than they've already done involving other properties of relations). The entire document still needs a good editing. Some parts are still rough. A most notably rough one is the first, on talking about languages.

Evaluation

It is a challenge toto teach this course using Lean 4 for significant net advantage over more traditional courses. The hard problem, now cracked I hope, was reduce the added burden imposed by conceptually irrelevant aspects of Lean sufficiently to not impair students' capacity to learn the underlying concepts. At this point I believe that we can teach DMT1 using this kind of technology to the significant net advantage of our students, while covering all the essential concepts of any DMT1 class without undue haste.

Use for DMT1

A conservative offering could cover chapters on propositional logic, arithmetic, and induction for arithmetic function definitions, then skip theory extensions and SMT, and proceed to predicate logic, set theory, before a final return to induction generalized to proof construction. A possible capstone would be on termination via well founded relations, as they now have full command of the concept of properties of relations. I plan to reduce coverage of SMT by 80% on the next offering to have more time for this stuff at the end.

Student Paths

From here, advanced courses in several areas are possible at both undergraduate and graduate levels: cyber-physical systems, programming language design and implementation, verification, provable security, machine learning (e.g., see AlphaProof), robotics, quantum computing, etc.

Tool Paths

Working knowledge of the concepts of this course will provide students with easy access to understanding the concepts underpinning and the use of a broad range reasoning technologies. Dafny and Z3 are good next steps and there are surely others. A small enhancement to this course would be a much shorter ”highlight” on Z3 then another on one Dafny, just to show that there's a whole new world of reasoning systems out there.

Research Paths

- Natural experiments potentially accessible (IRBs)
- Constructive mathematical concept inventories

There are surely problems and opportunities for improvement, in concept and presentation, here. Reach out if you wish.

Invitation

If you with to discuss, or alert us to possible issues, please do email me with DMTL4 in the subject line. It's my last name at Virginia, Edu.

© Kevin Sullivan 2024-2025.

Chapter 1

Languages

Computer science is deeply connected with the concept of languages, particularly formal languages, which are languages where the valid forms of expressions are precisely specific, unlike so-called natural languages, such as English, where one typically has greater expressive freedom.

In this course, we will distinguish between two varieties of formal languages, in turn. Languages in the first class are mostly intended to support the specification of practical *computations*. These are the *programming* languages. The languages in the second class are weighted toward the expression of propositions, or claims about some work, and reasoning about them.

Elements

A language consists of several essential components that define its structure and meaning. These include:

- **Syntax**: The formal arrangement of symbols and symbolic expressions.
- **Semantic Domains**: The entities to which symbols and expressions refer, commonly referred to as denotations.
- **Interpretations**: The mappings that relate symbols to their denotations.
- **Semantics**: The relations that map entire expressions to their corresponding denotations.

Understanding these elements is fundamental to both natural and formal languages, as they provide the foundation for meaning and interpretation.

Moods

Languages can be classified by their **moods**, which reflect different communicative intents:

- **Declarative (Indicative)**: Used to assert statements about reality (e.g., "It is the case that ...").
- **Optative**: Expresses a desired state of affairs (e.g., "I require it to be the case that ...").
- **Imperative**: Issues commands to achieve a certain action (e.g., "Make it the case that ...").
- **Conditional**: Specifies a dependency between conditions and outcomes (e.g., "If X is true, then it is the case that ...").
- **Subjunctive**: Expresses counterfactuals or uncertainty (e.g., "If only it were the case that ...").
- **Exclamatory**: Conveys strong emotions (e.g., "Oh my!").
- **Interrogative**: Seeks information through questioning (e.g., "Is it the case that ...?").

Purposes

Different languages serve different functional purposes, including:

- **Human-Human Communication**: Facilitating communication between people (e.g., English, Mandarin).
- **Human-Machine Communication**: Enabling humans to issue instructions to machines (e.g., Java, Python).
- **Machine-Machine Communication**: Standardized formats for automated data exchange (e.g., JSON, XML).
- **Automated Reasoning**: Languages designed for formal reasoning, program execution, and proof verification.

Natural vs. Formal

Natural Languages

Natural languages—such as English, Spanish, and Mandarin—evolve to maximize ease of communication among humans. However, they introduce challenges in rigorous reasoning due to inherent properties:

- **Ambiguity**: A single phrase may have multiple interpretations (e.g., "Shoes must be worn, dogs must be carried").
- **Imprecision**: Some instructions lack formal specificity (e.g., "Keep a reasonable distance from the next car").
- **Computational Complexity**: While natural languages have historically been difficult to process computationally, large language models (LLMs) are changing this dynamic.

Formal Languages

Formal languages—such as those used in logic, mathematics, and programming—are designed for precision and mechanical reasoning. Their primary benefits include:

- **Rigor**: Supporting exact definitions and structured reasoning.
- **Computability**: Enabling automated processing and verification.
- **Application Scope**: Used in programming, specification, verification, and mathematical proofs.

Imperative vs. Declarative

Imperative Languages

Imperative languages define step-by-step procedures for solving problems. These languages execute commands that transform mutable states and perform I/O operations. A classic example is a Python implementation of Newton's method for computing square roots.

While imperative languages enable efficient execution, they often lack expressiveness compared to declarative approaches.

Declarative Languages

Declarative languages emphasize expressiveness over step-by-step execution. Key characteristics include:

- **Syntax and Semantics**: Clearly defined structure and meaning.
- **Domains, Interpretations, and Evaluation**: A well-defined mapping between symbols and their interpretations.
- **Models**:\n - **Validity**: An expression is valid if it is true under all interpretations.
  - **Satisfiability**: An expression is satisfiable if at least one interpretation makes it true.
  - **Unsatisfiability**: An expression is unsatisfiable if no interpretation makes it true.
- **Expressiveness vs. Solvability Trade-offs**: More expressive languages tend to pose greater computational challenges.
- **Decidability**: The problem of determining the validity of arbitrary expressions may be undecidable in some cases.

Formal Language Case Study: Propositional Logic

A classic example of a formal language is **propositional logic**, which serves as a foundation for automated reasoning.

Syntax

Propositional logic consists of:

- **Literals**: Atomic Boolean values (e.g., `true`, `false`).
- **Variables**: Symbols that represent Boolean values (e.g., `P`, `Q`).
- **Operators**: Logical connectives such as `AND`, `OR`, `NOT`, `IMPLIES`.

Semantic Domain

Propositional logic is grounded in **Boolean algebra**, where expressions evaluate to either `true` or `false`.

Interpretations

- **Fixed Symbols**: Operators map to specific Boolean functions.
- **Variables**: Assigned Boolean values in a given interpretation.

Semantic Evaluation

Expressions are evaluated within a given interpretation, determining their truth values.

Properties of Expressions

- **Validity**: An expression is valid if it evaluates to `true` in all interpretations.
- **Satisfiability**: An expression is satisfiable if there exists at least one interpretation where it evaluates to `true`.
- **Unsatisfiability**: An expression is unsatisfiable if it evaluates to `false` in all interpretations.

Computations in Propositional Logic

Several computational tasks arise in propositional logic:

- **Model Finding**: Identifying an interpretation that satisfies a given expression.
- **Model Checking**: Verifying whether an interpretation satisfies a given expression.
- **Validity Checking**: Determining whether an expression is valid.
- **Counterexample Finding**: Identifying interpretations that falsify an expression.

Conclusion

Languages—whether natural or formal—play an essential role in communication, computation, and reasoning. While natural languages prioritize human usability, formal languages enable rigorous expression and mechanical processing. The study of formal languages, including logic and programming languages, continues to shape advancements in computing, verification, and artificial intelligence.

Chapter 2

Propositional Logic

2.1 Syntax

2.2 Syntax

```
namespace DMT1.Library.propLogic.syntax

structure Var : Type where
  mk :: (index: Nat)
deriving Repr

inductive UnOp : Type
| not : UnOp
deriving Repr

inductive BinOp : Type
| and
| or
| imp
| iff
deriving Repr

inductive Expr : Type
| lit_expr (from_bool : Bool) : Expr
| var_expr (from_var : Var)
| un_op_expr (op : UnOp) (e : Expr)
| bin_op_expr (op : BinOp) (e1 e2 : Expr)
deriving Repr

open Expr

notation:max " ⊤ " => (Expr.lit_expr true)
notation:max " ⊥ " => (lit_expr false)
notation:max "{" v "}" => (var_expr v)
notation:max "¬" p:40 => un_op_expr UnOp.not p
infixr:35 " ∧ "  =>  Expr.bin_op_expr BinOp.and
infixr:30 " ∨  "  => Expr.bin_op_expr BinOp.or
infixr:20 " ↔ " => bin_op_expr BinOp.iff
```

```
infixr:25 " ⇒ " => bin_op_expr BinOp.imp

end DMT1.Library.propLogic.syntax
```

2.3 Examples

2.4 Semantics

2.5 Interpretations

Chapter 3

Model Theory

THIS SECTION IS UNDER CONSTRUCTION.

The semantics of propositional logic enable the evaluation an expression under an interpretation (for it). As we've seen, an interpretation in propositional logic is in the form of a *function* from the variable expressions in the expression to the Boolean values that the interpretation is said to *assign* to them.

Interpretations as World Representations

There's another way to see the interpretations for a given proposition, where each one specifies one specific possible *world* (or *state of affairs*), within which the truth of the proposition is to be evalauted.

Example: The Dog is Dandy or The Cat is Comfy

Suppose we use *D* to stand for the proposition, *the dog is dandy*, and *C* for, *the cat is comfy*. Now consider the proposition, *C or D*. There are four worlds within which it can be evaluated. The proposition, understood as asserting that at least one of the conditions, *C* or *D*, holds in a given world. This statement is true in only three of the four worlds, exluding as a counterexammple the one where neither is true.

Propositions as Specifying Properties of Worlds

We can view our *or* expression as specifying the subset of all of those possible (here four) worlds that *have the property* of having at least one of *D* or *C* being true in that world. We thus have a first indicition of a notion of model theory. It has to be about models of propositions.

Model Worlds and Counterexamples Worlds

We call a world in which a proposition is true a
model for the proposition, and a world in which it's not true a *counterexample*.

Generalizing from Propositional Logic

At the heart of all of this is the idea that when syntactic expressions have elements with late-bound meanings, we need to provide additional structures, descriptions of specific worlds, states of affairs, within, or under, which evluation can be performed. Moreover, when a proposition is true within a world, that world is called a model„ and when false, it's called a counterexample.

Propositional Logic as a Special Case

In propositional logic, these structures are just our interpretation functions. Other logics use other structures. Nevertheless, across many logics, the structure over which a proposition is evaluated can be understood as specifying a particular world, or state of affairs, *within which* the truth of a given proposition can be evaluated.

Validity as A Central Concept in Logic and Maths

Now we come to a truly central idea. Sometimes, we want a proposition to defines not so much a property of certain selected worlds, but to express a more general form of reasoning that we expect that to be valid in *all* possible worlds.

An an example, the proposition *C or D* is *not* valid in propositional logic because there is one world where it's false: where both propositions are false. On the other hand, we can reasonably accept the idea that if *C and D* is true, then *C* is: that is. $C \boxed{?} D \boxed{?} C$. Let's call this expression, *e*.

We can easily see that *e* is valid by showing that it's true in in all four possible worlds. We form each of the four interpretations, then evaluate *e* under each of them, to find that *e* is *alwasy* true.

- when w := (C := $\boxed{?}$, D := $\boxed{?}$) *e** is true
- when w := (C := $\boxed{?}$, D := $\boxed{?}$) *e** is true
- when w := (C := $\boxed{?}$, D := $\boxed{?}$) *e** is true
- when w := (C := $\boxed{?}$, D := $\boxed{?}$) *e** is true

As is true in all worlds, it's said to be *valid*. We can now rightfully say that $C \boxed{?} D \boxed{?} C$ is not only a *valid* proposition, but a *theorem*, or that it's a reasonable, rational principle for reasoning under any any circumstances, in all worlds.

When we prove that a proposition in mathematics is a theorem, we are proving that it's valid. We can thus reasonably say it is a theorem that if *C* and *D* are any propositions then $C \boxed{?} D \boxed{?} C$ is always true. It's valid. It's a theorem (in propositional logic).

A useful logic is one that defines a set of such rules for reasoning that are both valid individually and that when used together can never produce a contradiction except from assumptions that are themselves false.

Example

Now would be a good time to revisit the examples of syntactically correct propositions from a few chapters back. The ones we picked are mostly valid, and in fact form the basis for reasoning in many related logics. Moreover the collection of the valid propositions will provide us with a partial foundation for reasoning in predicate logic, where that can be infinite spaces of worlds, and where validity can no longer be assured by semantic evaluation of a finite number of worlds.

Exercise: Determine which of the syntactically correct propositions in the bare code file are *not* valid, and for each of these expressions, give a counterexample in the form of an interpretation as a set of variable to value bindings as used just above.

Further Study and Next Steps

We've now given as much of an introduction as we can in this setting to the topic of model theory in teneral. Interested students might look at the *Kripke structure* over which propositions in some temporal logics are evaluated.

The Rest of This Chapter

For the rest of this chapter, we will focus on model theory for proposition logic, only. We will formally specify what validity means by brute for evaluation of a given expression over all possible inpretations for it and then

summing of the results.

In addition to being valid, we will also formally being *satisfiable* (having at least one model), and being *unsatisfiable* as having no models at all. We will then define procedures for checking whether all worlds are models (validity), whether at least one world is a model (satisfiability), or whether no worlds satify an expression to check for its being *unsatisfiable*.

Our discussion is already getting interesting, in that we've now viewed interpretations as representations of different worlds, propositions as specifying properties that a given world might more might not have, and now we're talking about *properties of propositions* (of being valid, satisfiable, or unsatisfiable). The rest of this chapter formalizes these ideas in executable forms, providing us with tools for automated reasoning about whether *any given proposition* has or lacks any of these properties.

Exercise

How many worlds are there in the *space of worlds* contemplated by an expression in propositional logic with n distint variable expressions?

3.1 Models

3.2 Truth Tables

3.3 Counterexamples

3.4 Propertiesx

3.5 Validity

Chapter 4

Arithmetic

In this chapter we adapt the what we've learned in the chapter on propositional logic syntax and semantics to specify the syntax and semantics of a simple language of *natural number arithmetic*. The language we specify here will have two distinct kinds of expressions.

An expression of the first kind of expression evaluates to a natural number. Consider $2 + X$, for example. The operator symbol, $+$, tells us that this expressions has a natural number as its semantic meaning: namely, the sum of *2* and whatever (now numberic) value X has under a given interpretation. We will refer to $+$ and similar symbols as arithmetic *operators*, and to expressions that use them as arithemtic *operator expressions*.

An expression of the second kind will evaluate not to a numeric value but to a Boolean,. Consider $2 < X$ as an example. The standard symbol, $<$ tells us that we can expect this expression to evaluate to a Boolean value: *true* if *2* is less than whatever value X evaluate to, and to *false* otherwise. We will refer to symbols such as $<$ as *predicates*, and expressions such as $2 < X$ as *predicate expressions*.

We will use the phrase *arithmetic expression* to refer to either an operator or a predicate expression in our little language of natural number arithmetic.

One of the main insights that you can expect to take away from this chapter is that you already knew how to formally define a language such as this one by simple adaptation of the lessons of the preceding chapter on the syntax and semantics of propositional logic.

4.1 Domain

4.2 Syntax

4.3 Semantics

4.4 Induction

4.5 Examples

Chapter 5

Theory Extensions

5.1 Domain

5.2 Syntax

5.3 Semantics

5.4 Examples

5.5 Satisfiability Modulo Theories

Chapter 6

Induction

Chapter 7

mathlib

Lean's mathlib is where the mathematical community's formalizations of lots of different parts of mathematics live. There are riches here, for almost any application.

The upshot for you at this point is that, once you master the next part of the course, on predicate logic in Lean 4, you can pick nearly any area of mathematics and reason in it, now with the full support of Lean, its infrastructure, and vibrant community.

The great thing is that, unlike first-order theory, Lean admits the practical, abstract, expressive, and verified embedding of an incredible diversity of logical and mathematical structures into its foundational logic, formulated in the abstract concepts of the mathematical domains, and made much more useful with concrete notations appropriate to the particular field being formalized. In Lean, one can speak easily about such things as properties of relations, or *real* real numbers. Neither is possible in the first-order that we mostly teach in traditional courses.

Make no mistake, first-order predicate logic with first-order theory of sets and relations is still an utterly indispensable language. It's syntax is used in many important reasoning systems, incluiding Z3, Dafney, Alloy, and others. But with knowledge of Lean, you *also* have immediate access to this *incredible* treasure trove of verified mathematical knowledge, *and* it's nearly trivial to understand first-order logic once you have seen the bigger picture.

If you're interested in a visual overview of areas of mathematics that Lean 4's mathlib provides, the overview of mathlib in Lean 3 is the best right now. The Lean 4 version is evolving and contributions are welcome.

Chapter 8

Predicate Logic

In propositional logic, syntactic variable expressions are interpreted only over sets of Boolean state values.

- PressureTooHigh /\ ORingsFailing
- (GreenLight \/ RedLight) /\ ~(GreenLight /\ RedLight)

Limited Expressiveness of Propositional Logic

Within the logic one can thus only speak about real world situations in terms of sets of Boolean variables and their values. The choice of propositional logic as a reasoning language limits one to representing real-world conditions in these very spartan, sometimes inadequate, and sometimes misleading terms.

As an example, suppose you want to reason about the safety of a driving vehicle. It has a brake and a steering system. Each is deemed 70.5% likely to be operational. That's high (by some low standards), so suppose we model this world in Boolean terms as { BrakeOk := true, SteeringOk := true }.

Now of course we want both brakes and steering for safety, so we evaluate BrakesOk /\ SteeringOk over this structure (intepretation) and get *true*. We might make the mistake of believing that the result soundly reflects reality. If the threshold for being Ok is a greater than 50% chance of success, this condition still doesn't hold: assuming that the failures are independent, we find that the combination of the probabilities yields a likelyhood of the overall *system* being ok is less than 50%.

Enhanced Expressiveness of Predicate Logic

A way out of this bind is to pick a more expressive logic: one that enables us to represent worlds in richer terms so that we might be enabled to reason about real worlds more effectively. The varieties of predicate logic provide such improved expressiveness.

In a predicate logic, we can speak not only in terms of Boolean variables and values, and the usual operations on them, but also in terms of *sets* of objects, *relations* between objects, properties (via predicates) of objects, *functions* that take objects as arguments and return them as results, and about whether all, or respectively at least one, of the elements of a given set has a given property.

It's on the foundation of a simple predicate logic that most of contemporary mathematics rests. To give a sense how this could work, note that one can encode the natural numbers as: either the empty set, encoding zero, or as a set containing the set representing a one-smaller number. The nesting level represents the value, just as for *Nat* in Lean.

The larger point here is that the choice of any given logic in which to reason about the world brings with it a form of structure in terms of which one can represent specific states of the real world, over which expressions are evaluated.

Semantic Domains for Predicate Logic

In particular, predicate logic brings along with it a much richer form of structure (than vector of Boolean) that one use to represent real world conditions of interest. A set of possible world situations no longer has to be encoded just as a vector of binary Boolean values. We can represent a world now in essentially *relational* terms. Predicate logic is the core theory underpinning relational databases. Our world state representation structures can now have:

- *objects* representing corresponding objects in the real domain (e.g., "Tom", "Mary")
- *functions* from objects to objects in the real world (e.g., motherOf Tom = Mary)
- *relations* among objects in the real world (e.g., youngerThan Tom Mary)
- *sets of objects* (e.g., { "Tom", "Mary" }) [unary relation]

To know how to use predicate logic effectively, it really helps to think about how you want to represent the space of all of the possible worlds about which you might speak, and how to represent specific individual worlds in that space. Think in relational terms. @@@ -/

/- @@@ The syntax of predicate logic is then set up to provide nice ways to talk about such world representations. The syntax provides *constant* symbols (referring to fixed domain objects); (free) *variables*, also interpreted as referring to objects; *function* symbols (and arguments, interpreted as referring to functions in the world); and *predicate* symbols refering to relations in the domain. Predicate logic inherits the connectives of propositional logic, adds two kinds of quantified expressions, and puts it all together into a syntax of *well formed formula* in predicate logic.

Limited Expressiveness of First-Order Predicate Logic

The variant of predicate logic taught in typical DMT1 courses is called first-order predicate logic. The logic of Lean is a higher-order logic. The difference is in the generality of what can be expressed in each of these logic.

To see the difference concretely, let's consider what properties a *friendOf* relation should have on some new social network. It could be *symmetric*, as it is on *FB*; or one might prefer for it not necessarily to be symmetric. Maybe a *follows* relation would be better. I follow Bill Gates but he doesn't follow me.

In first-order logic we can use the *universal quantifer* syntax to specify that our *friendOf* relation is (to be) symmetric. We can write this as ∀x∀y(*friendOf(x,y)*→*friendOf(y,x)).* The variables, *x* and *y*, are *bound* by the quantifer to range over all objects in the semantic domain. The predicate after the comma then checks whether all such pairs of objects satisfy the following predicate.

A major restriction on expressiveness imposed by first-order logic is that quantified variables can be range only over objects. One cannot quantify over all sets of objects, functions, or relations. In first order theory we can express the idea that some particular relation is symmetric.

What we can't express in first-order logic is the concept of symmetry as a generalized property that *any* given binary relation on *any* set of objects of *any* kind might or might not have. And yet being able to reason fluently with concepts at this level of generality is essential for any literate mathematician. in terms of We cannot express the *property* of a relation of *being symmetric,* a crucial degree of mathematical generality*, in first order theory because we cannot talk about *all relations.*

The Enhanced Expressiveness of Higher-Order Logic in Lean

The most crucial property of Lean for this course, and for research worldwide on the formalization of advanced mathematics, is that Lean implements a higher-order logic in which you can express concepts of great gener-

ality by quantifying not only over elementary values but over such things as types, functions, propositions and predicates, and so forth.

As an one example, in higher order predicate logic in Lean, one can define *generalized* properties of relations: for example the property of a binary relation r on a set s of values of some type T that r relates any two values in one direction only if it it also relates them in the other order.

```
def symmetric :=
∀ (T : Type)
  (R : T → T → Prop)
  (t1 t2 : T),
r t1 t2 ↔ r t2 t1
```

That is, for any type of objects, *T* (quantification over types), and for any binary relation, *R,* on values of this type (quantification over relations), what it means for *R* to be symmetric is that for any two objects, *t1, t2* of type *T* (a first-order quantification), if *R* relates *t1* to *t2* then it also relates *t2* to *t1*.

We can then *apply* such generalized definition to any particular binary relation on any type of values (let's say, *isFriend*, on values of type, *Person*) to derive the proposition that *that* relation is symmetric. The expression, *symmetric Person isFriend*, would then reduce to the first-order proposition, *? t1 t2 : Person, *isFriend t1 t2 <-> isFriend t2 t1*.

For the working mathematician it's crucial to be able to reason and speak in terms of such abstract and generalized properties of complex things. We speak of relations being symmetric, reflexive, transitive, well founded, and so on, without having to think about specific relations. We think of operation being commutative, associative, invertible, again in precise but abstract and general terms, independent of particular operations. In the first-order logic of the usual CS2: DMT1 class, that's just not possible. In Lean 4, it is completely natural and incredibly useful.

The result has been an explosion in the community of mathematicians using Lean 4 to formalize and verify theorems in many branches of advanced mathematics. That in turn has sparked deep interest in computer science in the use of Lean 4 for research and development in trustworthy computing, among many other areas. The rest of this chapter skips over first-order logic to introduce predicate logic through its higher-order, much more useful and relevant variant, in Lean.

8.1 Introduction

8.2 Propositions as Data Types

8.3 Propositions as Logical Types

8.4 Classical Reasoning

8.5 Predicates

8.6 Quantifiers

8.6.1 For All

8.6.2 Exists

8.7 Dependent Type Theory

Chapter 9

Sets Relations and Properties Thereof

Chapter 10

Abstract Algebra

Chapter 11

Type Heterogeneity

Getting Started

The DMTL4 book is largely generated from Lean 4 source. You can experiment with all of the examples in the VS Code IDE. First, follow the Lean 4 quickstart to install VS Code, Lean, and the Lean 4 extension. Second, install `git` and clone your fork of the repository. Next, open the repository in VS code. Finally, open the notes you wish to run. The book's Lean code can be found in the `code/DMT1/Lectures/` directory.

Building DMTL4

If you wish to generate the book yourself there are additional steps and dependencies you will need.

First, install `mdbook`.

Additionally, install `mdbook-toc`, `mdbook-mermaid` and `mdbook-image-size`. To do so, download the appropriate tarball for you system architecture and unpack it into a directory on your path.

Then, install the Haskell installer, `ghcup`.

Add `ghcup`'s bin directory (`$HOME/.ghcup/bin`) to your `PATH`.

Next, launch the TUI and install `ghcup`'s recommend Haskell version.

Lastly, if not already installed, install `make` on your platform.

At this point you should be able to build the book. From the top-level directory run `make all`. This will transform the code (in `code/DMT`) into markdown (in `src/DMT1`) for the book.

Finally, you can serve up the book for browsing with `mdbook serve --open &`. Conveniently, any changes to the `src` directory will cause the book to be rebuilt automatically.

Install Scripts

Manual installation instructions will be replaced with automated install scripts over time. We welcome any and all contributions to this effort.

Windows

Not available at this time.

macOS

The dependencies described above can be installed on macOS via homebrew with the following commands.

```
brew install mdbook ghcup

# Assuming x86_64. Tweak for apple silicon.
curl -L https://github.com/badboy/mdbook-mermaid/releases/download/v0.14.1/mdbook
-mermaid-v0.14.1-x86_64-apple-darwin.tar.gz | tar -xz -C $HOME/.local/bin
curl -L https://github.com/badboy/mdbook-toc/releases/download/0.14.2/mdbook-toc-0.14.2-x86_64-apple-darwi

# Assuming bash. Tweak if using a different shell.
echo 'PATH=$PATH:$HOME/.ghcup/bin' >> "$HOME/.bashrc"
```

```
ghcup install ghc recommended
ghcup set ghc recommended
```

Other

Note, Windows users using git via git bash and the command line should take care to select the appropriate shell in the VS Code terminal. It will do to set git bash as the terminal by changing Settings: Terminal > Integrated > Defafult Profile: Windows and selecting it there. Thereafter, launching a terminal in VS Code will launch one with a git bash shell. You can see what's possible there in the configuration section and eventually configure your own setup for terminal in VS Code.

Note, rust users can install the mdbook extensions with cargo.

Learning Resources

There are pretty good learning resources for Lean 4. Here we connect you to both written documentation for Lean 4 and to two online communities where you can follow the leaders in the field and ask questions, often getting answers very quickly.

Help Searching Mathlib for Relevant Definitions

- Mathlib Documentation
- Loogle online. Also see the Loogle VSCode extension.

Written Documentation on Lean 4 and Related Tools and Ideas

- Lean Language Site
- Lean Documentation Overview
- Functional Programming in Lean.
- Theorem Proving in Lean 4
- Meta-Programming in Lean 4
- Mathematics in Lean
- A Glimpse of Lean
- Type Checking in Lean 4
- Some Examples in Lean4
- Kevin Buzzard's Intro
- Lean for the Curious Mathematician (2023 edition)
- Learning Resources
- The Type Theory of Lean
- Type Checking in Lean
- Hitchhikers Guide to Logical Verification
- Useful Tactics for Beginners

Books on Closely Related Concepts and Tools (mainly Coq)

- Software Foundations
- The Mechanics of Proof
- Homotopy Type Theory, Chapter 1
- Survey of Logic Symbols
- Certified Programming with Dependent Types
- Software Foundations

Community Resources (Recommended to Join These Groups!)

- Lean Zulipchat

- Lean Discord Channel
- Programming Languages Discord Channel